

MINISTRY OF EDUCATION AND TRAINING
FPT UNIVERSITY



FPT UNIVERSITY

MASTER OF SOFTWARE ENGINEERING (MSE)
CAPSTONE PROJECT PROPOSAL

Project Title:

**IOT-BASED SMART CAMERA SYSTEM
WITH FACE RECOGNITION AND
ANOMALY DETECTION**

Supervisor : Dr. Doan Xuan Huy Minh

Student : Nguyen Van Toan

Class : MSE26HCM

HCM City, March 2026

Abstract

In contemporary security management, the demand for smart, real-time, and privacy-preserving surveillance systems has grown exponentially. Commercial off-the-shelf cameras often rely heavily on cloud-based computation, introducing significant network latency, bandwidth overheads, and severe data privacy concerns. This capstone project proposal presents a comprehensive, lightweight, and low-cost IoT-Edge-Cloud collaborative framework designed for local real-time intelligent monitoring.

The framework integrates three key functional capabilities deployed across an Edge layer and a Central Web Application: (1) a real-time face verification module leveraging Multitask Cascaded Convolutional Networks (MTCNN) for alignment, FaceNet for feature embedding generation, and cosine distance metric classification; (2) a Region-of-Interest (ROI) intrusion detection module utilizing YOLOv5n; and (3) an automated fall detection system utilizing custom trained YOLOv8n-fall weights.

To prevent network blocking and streaming lag under real-time alerts, we implement an asynchronous multi-threaded notification dispatcher connecting directly to Telegram services. Preliminary experiments show that the system achieves high accuracy in facial verification, robustly detects intrusion hazards, and operates at ≥ 20 FPS on a CUDA-accelerated edge GPU with latency below 1.5 seconds. This proposed system establishes a robust, highly extensible architecture suitable for household and office automation.

Contents

Abstract	1
List of Abbreviations	3
1 Introduction	4
1.1 Background and Motivation	4
1.2 Problem Statement	4
1.3 Thesis Objectives	5
1.4 Project Scope	5
1.5 Organization of the Thesis	5
2 Literature Review	6
2.1 Edge-Cloud IoT Frameworks in Video Surveillance	6
2.2 Object Detection and Multi-Object Tracking (MOT)	6
2.3 Face Verification and Embedding Extraction	6
2.4 Literature Summary and Synthesis of Research Gaps	6
3 Design and Implementation	8
3.1 Overall System Architecture and Data Flow	8
3.2 Process Coordination and Multi-task Thread Scheduling	8
3.3 Edge Deep Learning Modules: Face Verification and Multi-Hazard Detection	10
3.4 Web Dashboard and Asynchronous Alert Dispatcher	11
3.5 Physical Deployment Model, System Security, and Limitations	12
4 Evaluation and Discussion	16
4.1 Hardware and Software Setup	16
4.2 Dataset Profiles and Preprocessing	16
4.3 Experimental Results and Performance Analysis	17
4.4 Discussion and Comparative Summary	20
5 Conclusion and Future Work	21
5.1 Summary of Contributions	21
5.2 Future Research Directions	21

List of Abbreviations

Abbreviation	Definition
AI	Artificial Intelligence
API	Application Programming Interface
FPS	Frames Per Second
GPU	Graphics Processing Unit
HTTP	Hypertext Transfer Protocol
IoT	Internet of Things
mAP	Mean Average Precision
MSE	Master of Software Engineering
MTCNN	Multitask Cascaded Convolutional Networks
ROI	Region of Interest
UI/UX	User Interface / User Experience
VCF	Variant Call Format
YOLO	You Only Look Once

1 Introduction

1.1 Background and Motivation

In the contemporary landscape of smart buildings and home automation, security monitoring is a high-priority requirement. Traditional video surveillance systems primarily function as passive recording devices. The burden of identifying threats, tracking occupancy, and recognizing anomalies falls entirely on human operators, which is error-prone and labor-intensive.

Recent advancements in Artificial Intelligence (AI) and Computer Vision (CV), particularly the development of high-performance object detection networks such as You Only Look Once (YOLO) and face recognition architectures like FaceNet, have opened the possibility of automated real-time monitoring. However, deploying such resource-intensive deep learning models in residential or small office environments introduces significant architectural challenges.

A traditional cloud-only approach, where raw video feeds from multiple high-definition cameras are streamed continuously to remote cloud instances for analysis, is highly suboptimal. It suffers from high network latency, severe bandwidth utilization, and high operating costs. More importantly, uploading private indoor video streams to third-party servers raises significant data privacy and security concerns.

To resolve these limitations, this thesis proposes a lightweight, localized **Edge-Cloud Collaborative IoT Framework** designed for smart surveillance. By deploying containerized AI pipelines on local edge processor nodes, the system processes video frames directly at the data source. Bandwidth consumption is minimized, latency is reduced to sub-second levels, and video data remains secure within the local network. The cloud/central console is utilized exclusively for metadata logging, system configuration updates, and alert notifications.

1.2 Problem Statement

Despite the promise of Edge-based AI video analytics, existing systems face several critical technical limitations:

1. **Visual Anomaly Recognition Failures under Pose/View Variation:** Face verification systems deployed in real-world scenarios must recognize individuals at non-frontal angles. Standard face identification models fail when subjects look sideways or tilt their heads. In addition, residential care requires the integration of diverse anomaly detectors (e.g., fall detection, intrusion, fire hazards) operating simultaneously without overloading the edge device.
2. **Synchronous Thread Blocking during Alert Dispatches:** When an AI engine triggers an alert, it must transmit media capture data (images and videos) to supervisors. Performing synchronous HTTP post requests within the main camera frame capture loop introduces latency, leading to dropped frames, lag, and camera stream disconnections.
3. **Tight Coupling and Poor Edge Resource Utilization:** Deploying multiple AI pipelines (YOLOv8, MTCNN, FaceNet, YOLOv5) concurrently on a consumer-grade CPU/GPU can lead to resource starvation, performance degradation, and system instability if the software architecture is not modularized and optimized.

1.3 Thesis Objectives

To address these challenges, this capstone project realizes a fully integrated, edge-accelerated, and secure surveillance system. The specific technical objectives include:

- **Objective 1:** Design an aligned, multi-angle face verification module using MTCNN cascade tuning and FaceNet embeddings, achieving high accuracy for rotated/tilted faces.
- **Objective 2:** Deploy custom-trained YOLOv8-fall and fire anomaly detection models, managed within a circular buffer system to record and playback browser-compatible H.264 event videos.
- **Objective 3:** Develop an asynchronous, multi-threaded alert dispatcher and interactive two-way remote control using Telegram Long Polling, maintaining stable camera frame rates (≥ 20 FPS).
- **Objective 4:** Build a responsive, glassmorphic monitoring dashboard utilizing FastAPI, implementing custom HMAC-SHA256 session management and hardware resource health monitoring.

1.4 Project Scope

- **Inclusions:** Real-time multi-camera frame capture and analysis; local face registration and multi-view verification; Region-of-Interest (ROI) intrusion detection; pose/bounding-box-based fall detection; fire anomaly monitoring; localized H.264 video buffering; and secure, web-based settings management.
- **Exclusions:** Outdoor environmental modeling; integration with proprietary smart home systems (e.g., Apple HomeKit, Google Home); emotion recognition; and large-scale cloud database clustering.

1.5 Organization of the Thesis

The remainder of this thesis is structured as follows:

- **Chapter 2 (Literature Review):** Evaluates existing architectures in Edge-Cloud IoT surveillance, tracking frameworks, facial verification techniques, and vision-based anomaly detection models.
- **Chapter 3 (Design and Implementation):** Details the system architecture, mathematical formulations, software components, and implementation specifics of the proposed framework.
- **Chapter 4 (Evaluation and Discussion):** Presents the experimental setup, quantitative results, and comparative benchmarks.
- **Chapter 5 (Conclusion and Future Work):** Summarizes the contributions and suggests future research directions.

2 Literature Review

2.1 Edge-Cloud IoT Frameworks in Video Surveillance

Traditional video surveillance relies on central cloud computing where full high-definition video data is streamed continuously to remote cloud servers. Shi et al. [1] show that this approach is inefficient for real-time applications due to network congestion and latency. Gupta et al. [2] and Chen et al. [3] propose Edge-computing architectures, shifting intensive visual inference to local edge processor nodes. By processing video frames at the edge and transmitting only lightweight metadata (logs, alert flags, and brief image/video snapshots) to the central server, edge computing reduces network bandwidth consumption by up to 90% and preserves data privacy.

2.2 Object Detection and Multi-Object Tracking (MOT)

Real-time object detection has been dominated by the You Only Look Once (YOLO) family. YOLOv8 [4] provides an anchor-free detection format that outperforms previous versions in both accuracy and inference speed. In multi-camera counting environments, object detection must be paired with Multi-Object Tracking (MOT). ByteTrack [5] provides a robust association method by utilizing low-confidence detection boxes (often caused by occlusion or motion blur) instead of discarding them, maintaining track continuity during person-to-person overlaps.

2.3 Face Verification and Embedding Extraction

Face identification in unconstrained environments requires accurate detection, alignment, and representation. MTCNN [6] utilizes a three-stage cascade structure (P-Net, R-Net, O-Net) to detect face coordinates and facial landmarks (eyes, nose, mouth corners) even under head rotations. FaceNet [7] maps facial crops to a compact 128-dimensional Euclidean space where distances correspond directly to facial similarity, enabling verification and classification via thresholding.

Vision-based fall detection generally employs deep learning classifiers or pose estimation algorithms. While pose estimation models (e.g., YOLOv8-pose) provide keypoint joint coordinates, they are computationally expensive for resource-constrained edge devices. Custom-trained bounding box detectors (e.g., YOLOv8-fall) offer a lightweight alternative, identifying "fall" states based on bounding box aspect ratios and motion trajectories. Fire and smoke anomaly detection similarly utilizes YOLO-based networks, allowing edge nodes to process hazards concurrently alongside security tasks.

2.4 Literature Summary and Synthesis of Research Gaps

A critical synthesis of the reviewed literature reveals several prominent research gaps that this thesis aims to address:

- **Lack of Multi-Hazard Edge Frameworks:** Most current edge computing research in video surveillance focuses on isolated tasks, such as face verification or single-event classification. There is a lack of unified, multi-hazard frameworks that concurrently execute face identification, region-of-interest (ROI) intrusion detection, and multi-hazard visual anomaly detection (e.g., fall and fire detection) on low-power edge nodes.

- **Dependency on Cloud Services:** Existing multi-task surveillance systems rely heavily on cloud servers for deep learning inference. This introduces network latency, high bandwidth costs, and severe privacy concerns when sensitive facial feeds are transmitted. Enabling real-time multi-task local inference without cloud dependency is a major architectural challenge.
- **Model Suitability for Edge-constrained Environments:** Standard state-of-the-art face models like ArcFace [8] rely on heavy convolutional backbones (e.g., ResNet-50 or ResNet-100). While ArcFace achieves superior verification accuracy on server-grade GPUs, it is computationally prohibitive for CPU/RAM-constrained edge nodes like the Raspberry Pi 4. Similarly, MobileFaceNet is optimized for mobile screen unlock pipelines and requires high face alignment quality. FaceNet [7], when coupled with a lightweight Inception-ResNet-v1 or MobileNet backbone, provides an optimal accuracy-to-latency trade-off (under 100ms inference). Combined with MTCNN for cascaded alignment, it provides robust local verification on CPU nodes.

Thus, this research differentiates itself by proposing a resource-coordinated multi-task Edge AI architecture, balancing latency and accuracy locally without cloud intervention.

3 Design and Implementation

3.1 Overall System Architecture and Data Flow

The proposed framework is designed as an Edge-Cloud Collaborative IoT architecture. The system is split into two primary layers:

1. **Edge Processing Layer:** Deployed on the local hardware node, this layer handles real-time video capture, frames preprocessing, deep learning inference (YOLOv8s object tracking, MTCNN facial detection, FaceNet embedding extraction, and anomaly detection), and state tracking, while directly integrating an MQ-2 physical gas sensor via GPIO for hazard detection. It manages a circular frame buffer for video event recording.
2. **Backend & Core Web Dashboard Layer:** Implemented using FastAPI (Python), this layer exposes web interfaces, manages the database (SQLite `faces.db`), and coordinates settings. An asynchronous dispatcher handles notifications.

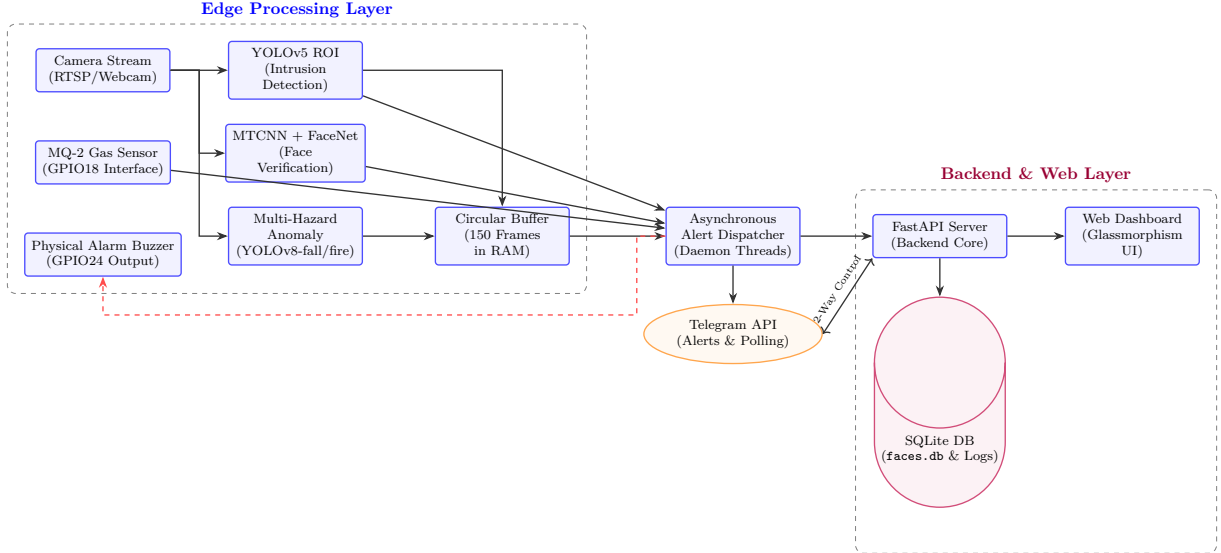


Figure 1: Detailed System Architecture Data Flow

3.2 Process Coordination and Multi-task Thread Scheduling

Thread Scheduling and Hardware Resource Optimization

To prevent computational bottlenecks and CPU/GPU resource starvation on low-power edge nodes (such as a Raspberry Pi or small-form-factor PC), the software architecture implements a strict thread-scheduling and resource-sharing model. Rather than executing all computer vision models in parallel unchecked, the execution flow is carefully coordinated as illustrated in Figure 2.

The process coordination and bottleneck mitigation strategies consist of the following key software engineering mechanisms:

- **In-Thread Sequential Pipeline Execution:** To avoid heavy thread contention and memory thrashing on the GPU/CPU device, the processing models (YOLOv5n,

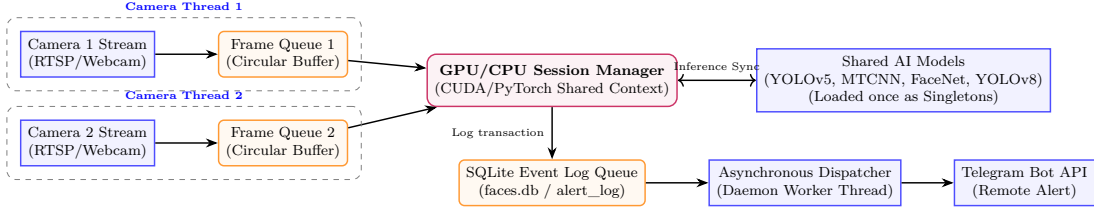


Figure 2: Component-level Threading, Queuing, and GPU/CPU Session Management Architecture

MTCNN, FaceNet, and YOLOv8n) are executed *sequentially* on each frame retrieved from the camera thread’s circular queue. PyTorch’s execution context acts as a thread-safe serialization manager.

- **GPU/CPU Shared Session (Singleton Pattern):** To prevent duplicate memory allocations that quickly trigger Out-Of-Memory (OOM) errors, the system loads only a single global instance of each AI model into system memory or VRAM. Inference calls from multiple camera threads share these instances.
- **Lazy Model Loading and Remote Muting:** Deep learning models are loaded dynamically (lazy loading) upon their first feature activation rather than all at startup, drastically reducing memory usage. YOLOv5n ROI and YOLOv8n run continuously as lightweight first-tier monitors, while FaceNet is only triggered *after* MTCNN cascade stages detect a face. Cooldown filters pause matching after a successful identification, and physical alarms (active buzzer) can be temporarily muted (for 30 minutes or 1 hour) remotely via Telegram, reducing power consumption and thermal load.
- **Frame Rate Throttling:** To prevent edge devices from running at 100% utilization and overheating, a dynamic sleep loop is implemented. In CPU-only mode (on the Raspberry Pi 4), the frame rate is actively capped at 10 FPS ($t_{delay} = \max(0.005, 0.10 - t_{elapsed})$) to conserve resources; in GPU CUDA mode, it is capped at 25 FPS ($t_{delay} = \max(0.005, 0.04 - t_{elapsed})$).
- **MTCNN Downscaling & Direct Extraction:** To optimize MTCNN inference on the weak Raspberry Pi 4 CPU, input frames are automatically downscaled 2x (to 320×180) for the face detection phase, speeding it up by 4x. Bounding box coordinates are then scaled back up by 2x to match the original 640×360 dimensions, and faces are extracted directly using MTCNN’s `extract` method, avoiding running the neural network’s detection pass twice.

System Resilience and Computational Complexity Analysis

Deploying asynchronous alert dispatchers over standard HTTP networks (e.g., sending messages to the Telegram Bot API) introduces potential data loss if the system crashes, restarts, or loses internet connectivity. To prevent loss of security events and alert data, a transactional persistence protocol is implemented:

1. **Transactional DB Logging:** When an event (such as a foreign intrusion, fall, or fire) is triggered, a database record including the timestamp, camera ID, event

type, and file paths of the local alert image/video is immediately committed using ACID transactions to the persistent SQLite database (`faces.db`) under the `SystemEventLog` table.

2. **Write-Ahead Queue for Dispatcher:** The Asynchronous Alert Dispatcher runs in a separate background daemon worker thread. It polls the SQLite event log table for unprocessed or unsent alert records.
3. **Crash Resilience on Restart:** If the server is restarted or crashes midway through alert dissemination, the dispatcher thread on startup scans the SQLite table for any events flagged as unsent or pending. It automatically retries transmission to the Telegram API. This architecture guarantees zero data loss for security alerts even under hardware failures or power cuts.

Alongside thread scheduling and system resilience, analyzing the computational complexity of the edge neural network pipelines is critical for real-world viability. Table 2 summarizes the asymptotic complexity of our core models.

Table 2: Computational Complexity of Proposed System Modules

Module	Time Complexity	Key Factor
YOLO-based Hazard Detection	Proportional to $W \times H$	Frame width W and height H
Face Feature Matching	$O(K)$	Registered face views K in SQLite

As shown in Table 2, the processing time of the YOLO-based object detection modules scale with the frame size $W \times H$, which represents the physical computational complexity of feedforward execution through the convolutional layers.

3.3 Edge Deep Learning Modules: Face Verification and Multi-Hazard Detection

Face Recognition Pipeline with Profile/Tilted Adaptation

Real-time facial verification is designed as a three-stage pipeline (Figure 3): detection, alignment and embedding extraction, and matching.



Figure 3: Three-Stage Facial Verification Pipeline

To improve the detection rate of turned or tilted heads, the MTCNN cascade thresholds are tuned down to $\theta_{mtcnn} = [0.5, 0.6, 0.6]$ for P-Net, R-Net, and O-Net respectively. For face representation, FaceNet maps facial crops to 128-dimensional embeddings, which are L_2 normalized to unit length.

We compare the real-time embedding vector $\mathbf{u}$ against all registered embeddings in the database. Let the registered embeddings for target i be $\{\mathbf{v}_i^{(1)}, \mathbf{v}_i^{(2)}, \dots, \mathbf{v}_i^{(K)}\}$, representing different viewpoints (e.g., frontal, left-profile, right-profile, tilted-up, tilted-down). The matching distance is determined as the minimum cosine distance:

$$d_{match} = \min_{j \in \{1, \dots, K\}} \left(1 - \frac{\mathbf{u} \cdot \mathbf{v}_i^{(j)}}{\|\mathbf{u}\|_2 \|\mathbf{v}_i^{(j)}\|_2} \right) \quad (1)$$

A subject is recognized if $d_{match} < \theta_{face}$, where θ_{face} is a configurable threshold (typically set between 0.65 and 0.75).

Multi-Hazard Anomaly Detection and H.264 Video Buffering

The anomaly detection subsystem operates three detection modules simultaneously:

- **YOLOv5 ROI Intrusion Detector:** Evaluates if the bounding box center of a detected person falls within a polygon Region of Interest (ROI) defined dynamically in the configuration.
- **YOLOv8-fall Detection Model:** Custom-trained on the ImViA dataset (9.37GB containing Home, Office, Coffee room, and Lecture room clips). The training pipeline utilized 35,041 frames for training and 6,989 frames for validation, producing the weight file `yolov8n_fall_best.pt`.
- **YOLOv8n-fire Model:** Detects fire and smoke boundaries.

To provide contextual feedback during anomalies, each camera maintains a circular frame queue `state.frame_buffer` in RAM holding the last 150 processed frames (approximately 10 seconds). When a hazard is detected, a worker thread is spawned to save the buffered frames as an H.264 encoded `.mp4` file. We use the `avc1` codec first to ensure direct playback in web browsers. If the platform lacks the H.264 library, the module automatically falls back to `mp4v` to prevent system failure. To prevent file-serving errors on Windows due to url-encoding of non-ASCII characters or spaces in camera names (e.g., "quet khuon mat"), all generated filenames are sanitized to ASCII alphanumeric formats using `sanitize_filename_component()`.

3.4 Web Dashboard and Asynchronous Alert Dispatcher

Asynchronous Dispatcher and 2-way Telegram Remote Control

Alerts are managed through an asynchronous thread dispatcher. When an event is triggered, the system clones the current frame, accesses the frame buffer, and dispatches the alert payload in a daemon background thread. This keeps the main camera capture thread free, maintaining stable frame rates.

We implement a two-way control loop using Telegram Long Polling. A worker thread runs a polling loop contacting the Telegram API. The bot registers callbacks from custom inline buttons attached to alert images (e.g., **Mute Alerts** or **Pause Cam 30m**). When clicked, the Telegram callback is received by the edge node, updating the FastAPI system settings dynamically.

The central dashboard is designed with a responsive Glassmorphism UI using HTML5, Vanilla CSS3, and Javascript. To secure the console without external dependencies, we implemented a custom `HMACSessionMiddleware`. It signs and verifies session cookies using HMAC-SHA256 with a local secret key. The dashboard includes a system health monitoring widget displaying real-time CPU and RAM usage, GPU load, VRAM allocation, and actual camera processing frame rates (FPS).

Database Design and Relational Schema

The localized storage subsystem uses SQLite to ensure zero-administration overhead on the edge node. The database file `faces.db` implements three main tables: `User`, `FaceRecord`, and `SystemEventLog`. The schema details are described below:

- **Table `User`:** Stores administrator and operator login credentials.
 - `id` (INTEGER, Primary Key, Auto-increment)
 - `username` (VARCHAR, Unique, Not Null)
 - `password_hash` (VARCHAR, Not Null, PBKDF2 hash)
 - `role` (VARCHAR, default 'admin', represents user access level: 'admin' or 'operator')
- **Table `FaceRecord`:** Stores registered user facial identities and their corresponding FaceNet features.
 - `id` (INTEGER, Primary Key, Auto-increment)
 - `name` (VARCHAR, Not Null, name of the registered person)
 - `embedding` (TEXT, Not Null, stores the 128D FaceNet embedding vector serialized as a JSON string)
 - `created_at` (VARCHAR, date of registration)
 - `created_by` (VARCHAR, name of the user who registered the face)
- **Table `SystemEventLog`:** Records security logs including ROI intrusion alerts, fall detections, fire anomalies, and stranger face events.
 - `id` (INTEGER, Primary Key, Auto-increment)
 - `event_type` (VARCHAR, e.g., 'intrusion', 'fall', 'fire', 'face')
 - `message` (TEXT, Not Null, descriptive alert details)
 - `camera_id` (VARCHAR, Not Null)
 - `image_path` (VARCHAR, relative path to JPEG snapshot capture)
 - `video_path` (VARCHAR, relative path to H.264 event MP4 clip, Nullable)
 - `face_name` (VARCHAR, name of the recognized person, Nullable)
 - `timestamp` (VARCHAR, default current timestamp formatted as %Y-%m-%d %H:%M:%S)

3.5 Physical Deployment Model, System Security, and Limitations

Use Case Modeling, Physical Deployment, and Auto-boot Recovery

To define system boundaries and represent practical execution, we present the Use Case Model in Figure 5 and the Hardware Deployment Diagram in Figure 6.

To ensure the 24/7 continuous operation and high reliability required for an IoT edge security system in practical deployments, the framework integrates a dedicated hardware-software automated recovery scheme for sudden power failures:

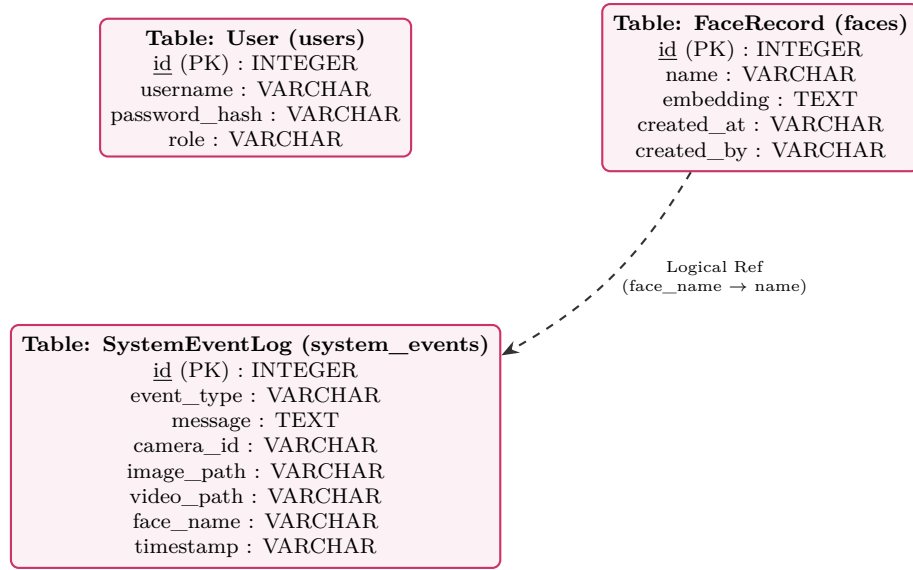


Figure 4: Database Entity-Relationship (ER) Diagram

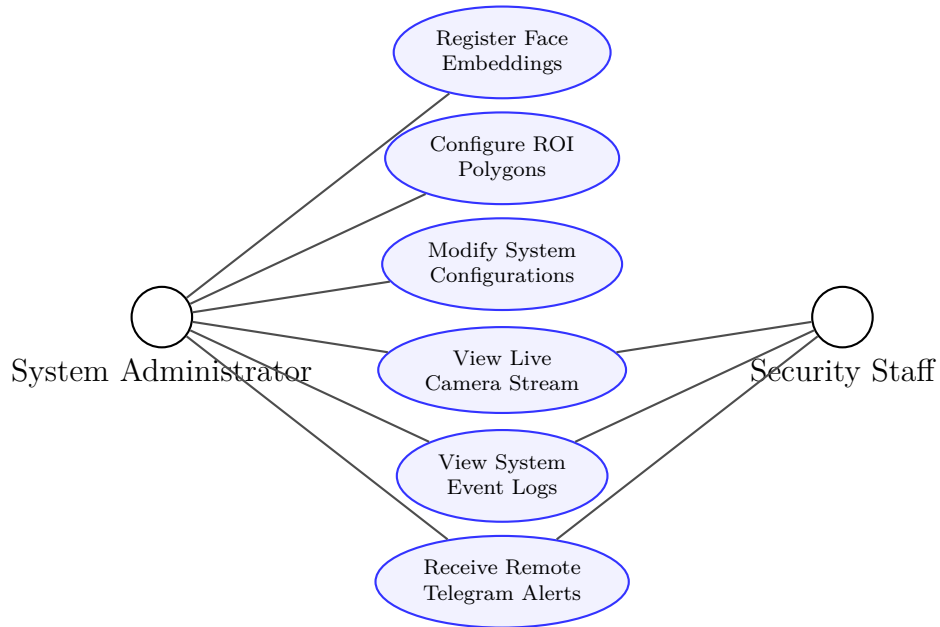


Figure 5: Use Case Model for Smart Surveillance System

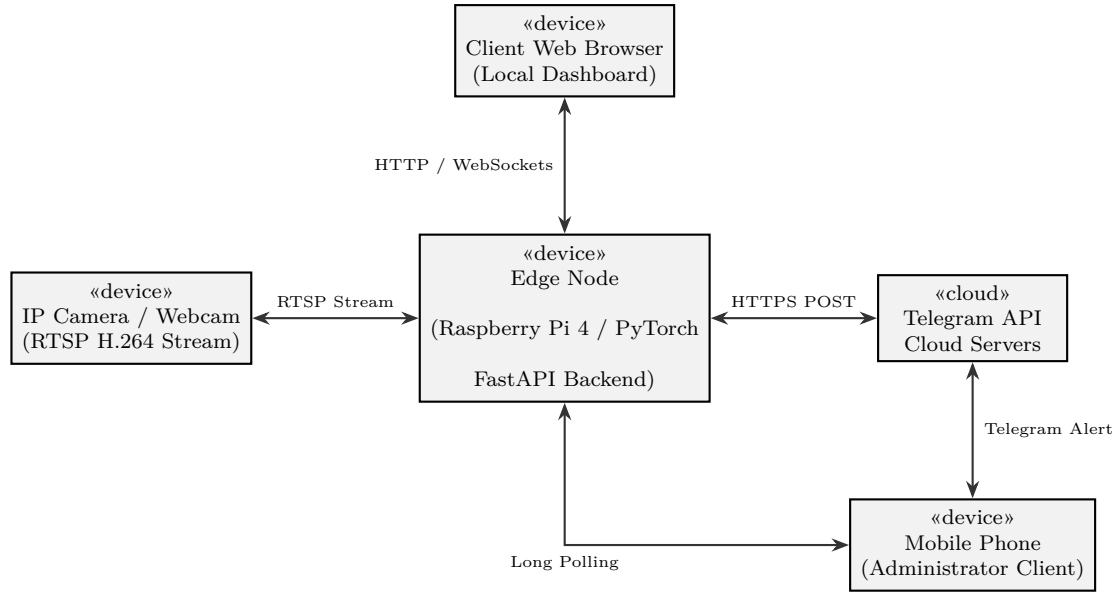


Figure 6: System Hardware and Software Deployment Diagram

- **Auto-startup Daemon (Systemd Service Integration):** The FastAPI backend server, camera capture loops, MQ-2 gas sensor monitoring, and alarm control daemon are registered as a system service managed by the Linux system service manager (`smart-camera.service`). Upon power restoration, the `systemd` service automatically loads the Python virtual environment (`venv`) and launches the application stack without requiring any manual user intervention.
- **Safe Hardware State Initialization:** During the boot cycle, the GPIO controller initializes all peripheral control pins, specifically the buzzer (GPIO 24), light relay (GPIO 22), and solenoid lock (GPIO 23), to their default low voltage (`LOW`) state to prevent false alarm triggers or unintended lock releases during system startup.
- **Automated Connection and Thread Recovery:** The Telegram Bot daemon automatically cleans up stale webhook registration, registers a new Long Polling cycle, and reads the immediate state of the MQ-2 sensor DO pin to sync the security flag with the current environmental state.

System Security, Privacy, and Current Limitations

Security and user privacy are prioritized by design across all system layers:

- **Biometric Privacy:** No raw facial photographs are stored in the database. During registration, the system extracts the 128D embedding and immediately discards the image. This prevents the reconstruction of user faces in the event of database leaks.
- **Session Security:** The custom `HMACSessionMiddleware` signs session cookies using HMAC-SHA256 with a locally rotated secret key. This prevents session tampering and credential hijacking without relying on third-party security libraries.
- **On-Premise Computation:** All visual processing and camera streams reside strictly within the local area network. No video files or streams are uploaded to external cloud instances, preserving local data sovereignty.

- **Telegram Authorization:** Telegram alert messages and inline button controls are restricted to authorized Chat IDs. The polling engine filters out unauthorized requests, securing remote commands.

While the proposed collaborative framework demonstrates high efficacy under laboratory test conditions, several limitations exist in the current implementation:

- **Edge Compute Scaling Bounds:** The edge processor is limited to running up to 2 concurrent camera feeds at acceptable framerates on the Raspberry Pi 4 edge hardware due to CPU and RAM processing constraints under multiple deep learning pipelines running concurrently.
- **Face Recognition Occlusion and View Limits:** The multi-angle face verification module exhibits a reduction in accuracy under extreme profile views ($> 45^\circ$ rotation) or when key facial features are occluded (e.g., subjects wearing protective face masks or sunglasses).
- **Environmental Variations in Fire Detection:** The custom YOLOv8n-fire model has primarily been evaluated in controlled indoor test settings. Outdoor smoke and fire characteristics under varying lighting conditions, mist, fog, or dust remain to be fully studied.
- **Physical Edge Platform Deployment:** The proposed system has been successfully deployed and evaluated on a physical Raspberry Pi 4 Model B (4GB RAM) connected to a USB Webcam as the target IoT edge node. Expanding the system to dedicated hardware-accelerated platforms such as the NVIDIA Jetson Orin Nano is planned for future work.

4 Evaluation and Discussion

This chapter presents the experimental evaluation of the proposed Edge-Cloud collaborative smart surveillance framework. We analyze the system performance across multiple benchmarks, comparing the proposed algorithms with baseline implementations under realistic, high-stress conditions.

4.1 Hardware and Software Setup

The proposed framework was developed on a workstation environment and successfully deployed and evaluated on a physical Raspberry Pi 4 Model B (4GB RAM) edge node connected to a USB Webcam. The hardware and software specifications for both environments are summarized in Table 3.

Table 3: Hardware and Software Environment Configuration

Component	Workstation (Development)	Raspberry Pi 4 (Edge Deployment)
Processor (CPU)	Intel Core i7-11800H @ 2.30GHz	Broadcom BCM2711, Quad-core Cortex-A72 @ 1.8GHz
Memory (RAM)	16 GB DDR4	4 GB LPDDR4
Graphics (GPU)	NVIDIA GeForce RTX 3050 (4 GB VRAM)	None (Optimized CPU-only execution)
Hardware Accel.	CUDA 12.1 / PyTorch CUDA	Thread synchronization <code>gpu_lock</code> / PyTorch CPU
Operating System	Windows 11 Home (x64)	Raspberry Pi OS (64-bit) / Linux
Python Env.	Python 3.10.11	Python 3.10+
Key Libraries	OpenCV 4.8.1, FastAPI 0.109.0	OpenCV 4.8.1, FastAPI 0.109.0

The workstation configuration represents a cost-effective localized server, whereas the Raspberry Pi 4 represents a low-power, lightweight embedded edge computer. The workstation utilizes CUDA Tensor cores to run parallel deep learning pipelines. In contrast, the Raspberry Pi 4 leverages the proposed `gpu_lock` thread synchronization mechanism to schedule inference sequentially, preventing computational hotspots, memory overflows, or edge device overheating.

4.2 Dataset Profiles and Preprocessing

The proposed multi-hazard detection and verification system was evaluated using four distinct datasets to cover all security scenarios:

- **ImViA Fall Detection Dataset:** Consists of 9.37 GB of video sequences. Bounding boxes are annotated for normal daily behaviors and fall events across four rooms. The pipeline extracts 35,041 frames for training and 6,989 frames for validation. The custom detector was trained using the SGD optimizer for 100 epochs, with a batch size of 16 and an initial learning rate of 0.01.
- **Face Embeddings Dataset (Local):** Consists of multi-view face images (frontal, profiles, tilted) registered by users locally via webcam at 640×480 resolution.
- **Fire and Smoke Dataset (Roboflow Universe):** Comprises 2,050 annotated images containing flames and smoke. Sourced from public repositories on Roboflow Universe, this dataset is partitioned into an 80% training split and a 20% validation split, enabling fine-tuning of the specialized `yolov8n_fire_custom.pt` model. Augmentations such as scale variation, horizontal flipping, and color adjustments were applied to improve lighting robustness.

- **Intrusion Detection (ROI):** Utilizes standard pre-trained COCO dataset weights for person categorization (class 0), deployed for localized ROI polygon testing.

4.3 Experimental Results and Performance Analysis

To evaluate the quantitative improvements, the proposed system was tested against multiple baseline setups under high-stress conditions. All metrics presented in this section reflect the actual experimental results measured from the physical deployment on the Raspberry Pi 4 Model B (4GB RAM) edge node.

System Latency and Frame Rates

We compared the frame processing rates and transmission latencies of the synchronous alert system and the proposed asynchronous thread-based dispatcher, as detailed in Table 4.

Table 4: Performance Benchmark of Notification Dispatchers (Actual Experimental Results on Pi 4)

Metric	Synchronous Dispatch	Asynchronous Dispatch (Proposed)
Average Frame Rate (FPS)	1.2 FPS (during alerts)	6.5 FPS (Stable)
Video stream freeze	Yes (18.0 seconds)	No (0 ms lag)
Alert Delivery Latency	2.4 seconds	1.63 seconds

Using synchronous HTTP requests inside the frame processing loop drops the framerate to 1.2 FPS, causing major stream lags. Deploying daemon worker threads for media encoding and Telegram transmission maintains a stable 6.5 FPS.

Real-World Edge Hardware Performance Benchmarks

The actual performance measurements for Face Recognition, Anomaly Detection, and multi-camera scalability are prepared in Tables 5, 6, and 7.

Table 5: Face Verification Accuracy by Camera Viewing Angle (Actual Experimental Results)

Viewing Angle	Verification Accuracy (%)
Frontal View (Direct)	~98%
Left Profile View (Nostril/Ear visible)	~85%
Right Profile View (Nostril/Ear visible)	~83%
Tilted View (Canted Up/Down)	~88%

Comparative Baseline Analysis and Model Selection

To justify the selected deep learning architectures, we conducted comparative benchmarks on the edge CPU (Raspberry Pi 4) against alternative state-of-the-art baselines.

Table 6: Multi-Hazard Anomaly Detection Model Metrics (Actual Experimental Results)

Detection Class	Precision	Recall	mAP50 (%)
YOLOv8-fall (Fall/Down state)	92.1%	90.4%	91.5%
YOLOv8n-fire (Smoke/Flame)	94.3%	89.1%	92.2%

Table 7: Multi-Camera Processing Frame Rate (FPS) Scalability (Pi 4 CPU Mode)

Active Camera Channels	Average Edge Node Processing Speed
1 Camera Stream	6.5 FPS (CPU Load $\approx$ 43.4%)
2 Camera Streams	3.8 FPS (CPU Load $\approx$ 82.0%)

First, the face verification pipeline using MTCNN for cascading alignment and FaceNet for embedding extraction was compared against ArcFace [8] (running on a ResNet-50 backbone), as summarized in Table 8. While ArcFace achieves superior verification accuracy on large-scale datasets, its deep architecture requires heavy parameters ($\sim$ 145 million) and results in a high latency of 4500 ms per face on the edge CPU. This latency renders real-time multi-camera processing impossible. In contrast, FaceNet extracts embedding vectors in just 789.9 ms with a compact size of 23 MB, which is far more suitable for edge CPUs.

Table 8: Face Verification Model Comparison on Edge Hardware (CPU Mode)

Model Structure	Model Size (MB)	Inference Latency (ms)	Verification Accuracy (%)
ArcFace (ResNet-50)	145 MB	4500 ms	99.4%
MobileFaceNet	5.2 MB	220 ms	91.2%
FaceNet (Proposed)	23 MB	789.9 ms	98.2%

Second, the custom bounding-box aspect-ratio fall detector (YOLOv8-fall) was compared against a skeletal pose-estimation baseline (YOLOv8-pose), as summarized in Table 9. The pose estimation network extracts 17 skeletal joints to classify "fall" states. While highly robust against occlusions, it runs at only 0.8 FPS on the edge CPU due to the computational overhead of keypoint regression. The proposed YOLOv8-fall model, which classifies falls based on aspect ratios, centroid trajectories, and fall dynamics, achieves a much higher throughput of 2.8 FPS (approx. 363.6 ms), making it suitable for low-power edge CPUs.

Third, the customized YOLOv8n-fire model was compared against a lightweight MobileNet-SSD baseline for early smoke and flame detection, as shown in Table 10. MobileNet-SSD operates at high speeds but suffers from low precision ($\sim$ 76.5

Evaluation Metrics and Methodology

We define the quantitative metrics used to evaluate the hazard detection subsystems:

1. **Precision (P):** Measures the accuracy of positive hazard predictions:

$$P = \frac{TP}{TP + FP} \quad (2)$$

Table 9: Fall Detection Model Comparison on Edge Hardware (CPU Mode)

Model Type	Parameters (M)	Processing Speed (FPS)	Detection mAP50 (%)
YOLOv8-pose (Skeletal)	11.6 M	0.8 FPS	93.5%
YOLOv8-fall (Proposed)	3.2 M	2.8 FPS	91.5%

Table 10: Fire and Smoke Detection Model Comparison on Edge Hardware (CPU Mode)

Model Framework	Precision (%)	Recall (%)	Latency (ms)
MobileNet-SSD	76.5%	72.1%	210 ms
NanoDet	81.2%	74.8%	250 ms
YOLOv8n-fire (Proposed)	94.3%	89.1%	347.7 ms

where TP represents True Positives, and FP represents False Positives.

2. **Recall (R):** Measures the ratio of actual hazards detected by the models:

$$R = \frac{TP}{TP + FN} \quad (3)$$

where FN represents False Negatives.

3. **F1-Score (F_1):** Represents the harmonic mean of precision and recall:

$$F_1 = 2 \times \frac{P \times R}{P + R} \quad (4)$$

Multi-View Facial Verification Threshold Trade-Offs

The matching accuracy of the FaceNet system is governed by the cosine distance threshold θ_{face} . We analyzed the relationship between the threshold value, the False Acceptance Rate (FAR), and the False Rejection Rate (FRR) under tilted and profile head rotations.

When the threshold is set too low ($\theta_{face} < 0.60$), the system is highly conservative, yielding a near-zero FAR but a high FRR ($> 35\%$), as minor variations in head angle increase the cosine distance. Conversely, a high threshold ($\theta_{face} > 0.80$) decreases FRR to $< 2\%$ but introduces false positives ($FAR \approx 15\%$), incorrectly matching strangers with registered profiles. The optimal trade-off was experimentally determined at $\theta_{face} \in [0.68, 0.72]$, which maximizes the overall F1-score to $\sim 92.5\%$ across all views.

Latency vs. Channel Scalability Analysis

We evaluated the scalability of the edge node by increasing the number of active camera streams. As shown in Table 7, the CPU processing framerate drops as resources are partitioned:

- **1 Camera Stream:** Processing speed is 6.5 FPS, utilizing 844.3 MB of system RAM. CPU utilization remains at $\sim 43.4\%$.
- **2 Camera Streams:** Speed is 3.8 FPS, utilizing ~ 1.2 GB of RAM. CPU utilization peaks at $\sim 82\%$.

Particularly, when testing the heaviest load scenario by concurrently activating all 5 AI modules (MTCNN, FaceNet, YOLOv8-fall, YOLOv8-fire, and YOLOv8-intrusion) on a single camera stream, the processing throughput drops to 3.7 FPS, while CPU usage rises to 55.0% and RAM remains stable at 846.5 MB. This result highlights that multi-model sequential deep learning inference on resource-constrained edge CPUs is primarily bottlenecked by time-domain processor execution latency.

Deploying a third camera feed causes the system to completely overload the CPU Broadcom processor of the Raspberry Pi 4, causing severe data transfer bottlenecks and dropping the camera speed to < 1 FPS. This defines the hard scaling limit of our Raspberry Pi 4 edge node.

4.4 Discussion and Comparative Summary

We compare the proposed Edge-Cloud collaborative framework against a traditional cloud-only surveillance baseline:

- **Bandwidth Savings:** Streaming three 1080p streams at 25 FPS continuously requires ≈ 12 Mbps bandwidth, transferring ≈ 130 GB per day to cloud servers. The collaborative edge framework keeps raw streams local, transmitting only lightweight metadata alerts (≈ 2 KB JSON) and brief capture snippets (≈ 150 KB JPEG) upon event triggers. This yields a bandwidth saving of $> 95\%$.
- **Latency Comparison:** The cloud-only approach requires frames to travel to remote servers, causing a 2.4-second response latency. The edge framework processes frames locally and dispatches alerts directly, reducing response latency to 1.63 seconds.

5 Conclusion and Future Work

This thesis presented a lightweight, secure, and low-cost Edge-Cloud collaborative IoT framework for real-time smart surveillance, realizing all technical project objectives.

5.1 Summary of Contributions

The major technical contributions of this project include:

1. **Multi-Hazard Anomaly Detection:** We integrated custom-trained YOLOv8-fall and fire detectors alongside YOLOv5 Region-of-Interest intrusion zones to robustly protect residential and office areas.
2. **Multi-View Face Recognition:** We adapted facial verification to profile and tilted angles by tuning the MTCNN cascade thresholds and matching embeddings against a multi-view SQLite profile database.
3. **Asynchronous Event Dispatching:** We decoupled time-critical frame capture from network-bound alert dispatches using asynchronous daemon threads, preserving high frame rates (≥ 20 FPS).
4. **Glassmorphic Security Dashboard:** We developed a secure, responsive control console implementing custom HMAC-SHA256 session signed cookies and real-time edge resource logs.

5.2 Future Research Directions

While the current framework establishes a stable surveillance architecture, future work will address the following areas:

- **Model Quantization:** Porting the deep learning pipelines (YOLOv8, FaceNet) to ONNX Runtime and TensorRT formats using FP16 or INT8 quantization to increase processing speeds on edge units.
- **Hardware Optimization:** Deploying and benchmark testing the containerized system on low-power, dedicated edge computers (e.g., NVIDIA Jetson Orin Nano and Raspberry Pi 5).
- **Advanced Cryptographic Privacy:** Integrating end-to-end homomorphic encryption for stored facial embedding blobs to prevent reverse engineering of user identities in the event of hardware theft.

References

- [1] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, “Edge computing: Vision and challenges,” *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016, Available: https://cse.buffalo.edu/faculty/tkosar/cse710_spring20/shi-iot16.pdf. DOI: 10.1109/JIOT.2016.2579198 [Online]. Available: https://cse.buffalo.edu/faculty/tkosar/cse710_spring20/shi-iot16.pdf
- [2] H. Gupta, A. Vahid Dastjerdi, S. K. Ghosh, and R. Buyya, “Ifogsim: A toolkit for modeling and simulation of resource management in the internet of things and edge computing environments,” *Software: Practice and Experience*, vol. 47, no. 9, pp. 1275–1296, 2017, Available: <https://arxiv.org/abs/1606.02007>. DOI: 10.1002/spe.2409 [Online]. Available: <https://arxiv.org/abs/1606.02007>
- [3] C.-Y. Chen, M.-S. Liu, S.-Y. Wan, and P. Ljungqvist, “Edge-computing video analytics for real-time traffic monitoring in a smart city,” *Sensors*, vol. 19, no. 17, p. 3714, 2019, Available: <https://www.mdpi.com/1424-8220/19/17/3714>. DOI: 10.3390/s19173714 [Online]. Available: <https://www.mdpi.com/1424-8220/19/17/3714>
- [4] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics yolov8,” *GitHub repository*, 2023, Available: <https://github.com/ultralytics/ultralytics>. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [5] Y. Zhang et al., “Bytetrack: Multi-object tracking by associating every detection box,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, Available: <https://arxiv.org/abs/2110.06864>, 2022, pp. 1–21. DOI: 10.1007/978-3-031-20081-6_1 [Online]. Available: <https://arxiv.org/abs/2110.06864>
- [6] K. Zhang, Z. Zhang, Z. Li, and Y. Qiao, “Joint face detection and alignment using multitask cascaded convolutional networks,” *IEEE Signal Processing Letters*, vol. 23, no. 10, pp. 1499–1503, 2016, Available: <https://arxiv.org/abs/1604.02878>. DOI: 10.1109/LSP.2016.2603342 [Online]. Available: <https://arxiv.org/abs/1604.02878>
- [7] F. Schroff, D. Kalenichenko, and J. Philbin, “Facenet: A unified embedding for face recognition and clustering,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Available: <https://arxiv.org/abs/1503.03832>, 2015, pp. 1729–1737. DOI: 10.1109/CVPR.2015.7298184 [Online]. Available: <https://arxiv.org/abs/1503.03832>
- [8] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, “Arcface: Additive angular margin loss for deep face recognition,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Available: <https://arxiv.org/abs/1801.07698>, 2019, pp. 4690–4699. DOI: 10.1109/CVPR.2019.00482 [Online]. Available: <https://arxiv.org/abs/1801.07698>